Université de Montpellier
Faculté des Sciences
Master 1 Informatique, parcours ICO (Intégration de compétences)

HAI704I — Architectures logicielles distribuées

TP 1 — Java RMI : cabinet vétérinaire

Enseignant responsable : Bachar Rima
Année universitaire : 2026–2027
Date : 22/10/2026

Étudiant :
Nom : VAROL
Prénom : Serdar
Numéro étudiant : 22112697
Parcours : ICO

1. Objectifs du TP 3
2. Architecture générale (étape A0) 3
3. Étape A1 — Un premier objet distant : l'animal 4
4. Étape A2 — L'espèce : une donnée qui traverse le réseau 4
5. Étape A3 — Le dossier de suivi : objet distant ou copie ? 5
6. Étape A4 — Le cabinet : un point d'entrée unique 6
7. Étape A5 — Créer un patient depuis le client 6
8. Étape A6 — Alertes : le serveur rappelle ses clients 7
9. Étape A7 — Ce qui traverse vraiment le réseau 8
10. Étape A8 — Une application utilisable : la CLI 8
11. Synthèse 9
12. Regard critique 10
13. Usage des outils d'intelligence artificielle générative 10
14. Références 12

1. Objectifs du TP

Ce TP consiste à construire, en Java RMI « pur » (sans framework), une application répartie pour un cabinet vétérinaire : plusieurs vétérinaires, depuis des postes clients distincts, consultent et modifient les patients et leurs dossiers de suivi hébergés sur un serveur, et sont prévenus automatiquement quand le nombre de patients franchit 100, 500 ou 1000.

Le fil conducteur est une question de conception : pour chaque donnée, doit-elle **rester sur le serveur** (objet distant, passé par référence via un stub) ou **être copiée chez le client** (objet sérialisable, passé par valeur) ? Huit étapes (A1 à A8) y répondent une à une, de l'objet distant minimal à une interface en ligne de commande utilisable.

2. Architecture générale (étape A0)

L'application est répartie en **trois projets indépendants**, sans `module-info.java`, compilés manuellement avec `javac` :

Projet	Rôle	Contenu
`common`	Contrat partagé	Interfaces distantes `IAnimal`, `IDossierSuivi`, `ICabinet`, `IObservateurCabinet` ; classes sérialisables `Espece`, `Chien`
`server`	Côté serveur	Servants `AnimalImpl`, `DossierSuiviImpl`, `CabinetImpl` ; lanceur `Server`
`client`	Côté client	`Main`, `ConsoleUI`, `ClientCabinet`, `ObservateurAlertes`, `ErreurCabinet`

Décision : le client ne dépend jamais du serveur. `server` et `client` n'ont que `common` dans leur classpath. Le client ne connaît donc que des interfaces ; il ne peut pas instancier `AnimalImpl`, ce qui l'oblige à passer par le serveur pour toute création (voir A5). Vérification : `grep -rn "vete.server" client/src common/src` ne renvoie rien.

Registre interne. Le serveur crée lui-même le registre (`LocateRegistry.createRegistry(1099)`) dans sa JVM et n'y publie qu'un objet, `cabinet` (avec `rebind`, plus tolérant que `bind` en développement). Il y a donc un processus serveur et un processus par client, lancés dans des terminaux distincts.

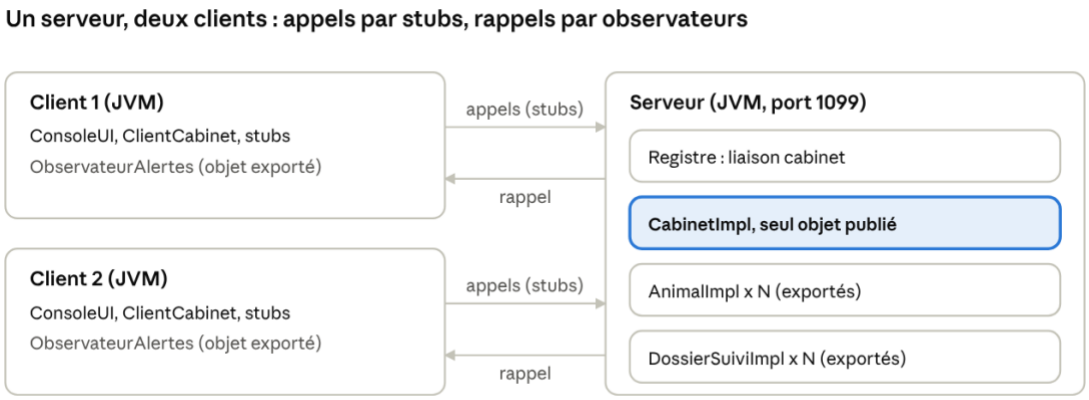


Schéma 1. Les clients appellent le serveur par des stubs ; le serveur rappelle les observateurs que les clients ont exportés (A6).

Point de contrôle 0. `java -version` annonce Java 21 (le sujet demande 17 ou plus). Les exemples `hello-rmi-single-project` et `hello-rmi-multi-project` (registre externe et interne) ont été exécutés ; les trois projets de l'application compilent.

3. Étape A1 — Un premier objet distant : l'animal

Implémentation.

- `common` : `IAnimal extends Remote`, dont chaque méthode (`getName`, `getProp`, `getRace`, `getEspece`) déclare `RemoteException`.
- `server` : `AnimalImpl extends UnicastRemoteObject implements IAnimal`. L'appel implicite à `super()` exporte l'objet.
- `Server` crée les patients, crée le registre sur le port 1099 et publie avec `rebind`.
- Le client fait un `lookup`, puis appelle les méthodes d'affichage.

Décision. Le registre est créé dans la JVM du serveur plutôt que lancé avec `rmiregistry` : un processus de moins à gérer, et le sujet accepte les deux.

Validation (point de contrôle A). Depuis un processus client séparé :

```
Nom : Moon
Proprietaire : Alex
Race : samoyed
Espece : chien
Classe du stub : jdk.proxy1.$Proxy0
Proxy dynamique ? true
```

Le client ne possède pas l'animal : il détient un **stub**, un proxy dynamique (`java.lang.reflect.Proxy`) généré à l'exportation, qui transmet chaque appel au servant dans la JVM du serveur.

4. Étape A2 — L'espèce : une donnée qui traverse le réseau

Implémentation. `Espece` (nom, espérance de vie) implémente `Serializable`. `IAnimal.getEspece()` la retourne ; aucune opération `setEspece` n'existe dans le contrat distant, donc l'espèce ne peut pas être modifiée à distance.

Décision — où doit vivre cette classe ? Dans `common`. Le client et le serveur doivent tous deux la connaître : le serveur pour l'envoyer, le client pour la reconstruire. Si elle n'était que d'un côté, la désérialisation échouerait (c'est exactement l'expérience de l'étape A7).

Validation (point de contrôle B). Le client récupère l'espèce, la modifie localement (`setEsperanceVie(99)`), puis la redemande :

```
[client]  identityHashCode = 1170794006
[client]  avant mutation : Espece(chien, 13 ans)
[client]  apres mutation locale : Espece(chien, 99 ans)
```

```
[client]  redemande au serveur : Espece(chien, 13 ans)
[serveur] identityHashCode = 1558492786
```

Les deux `identityHashCode` diffèrent : il existe deux objets Java distincts, le client manipule une **copie**. Le serveur renvoie toujours 13 ans et affiche le même `identityHashCode` à chaque appel : un seul objet côté serveur, une nouvelle copie côté client à chaque appel. C'est la différence entre « envoyer une photo de l'objet » et « envoyer un moyen de le joindre ».

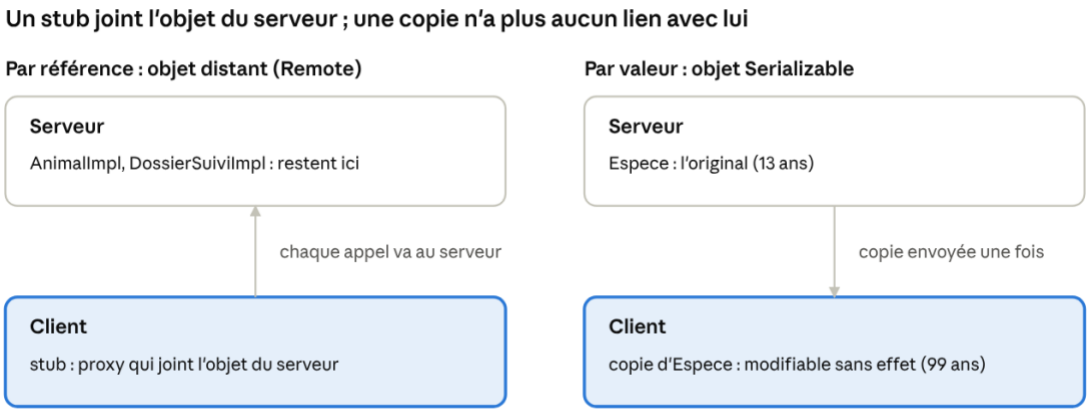


Schéma 2. Les deux modes de passage de RMI : `IAnimal`, `IDossierSuivi`, `ICabinet` et `IObservateurCabinet` voyagent comme stubs ; `Espece` (et `Chien`) comme copies.

5. Étape A3 — Le dossier de suivi : objet distant ou copie ?

Décision (la décision centrale du TP) : le dossier de suivi est un objet distant. Il reste dans la JVM du serveur ; le client n'en détient qu'un stub.

Justification. Plusieurs vétérinaires, donc plusieurs clients, doivent consulter et modifier le même dossier. Avec un objet distant, il n'existe qu'un dossier, côté serveur : toute modification est visible par tous. Avec un objet sérialisable, chaque client travaillerait sur sa propre copie et rien ne serait jamais partagé. Le choix découle donc de la question du partage, par opposition à l'espèce (A2), qui est une valeur immuable à distance et peut être copiée sans dommage.

Implémentation.

- `IDossierSuivi extends Remote` : `getEtatSante`, `setEtatSante`, `ajouterObservation`, `getObservations`.
- `DossierSuiviImpl` est le servant. Ses méthodes sont `synchronized`, car plusieurs clients peuvent l'appeler simultanément.
- `getObservations()` renvoie une copie de la liste (`ArrayList`, sérialisable) : le client ne peut pas modifier la liste interne du serveur.
- `IAnimal.getDossier()` retourne un `IDossierSuivi` : c'est un stub qui est transmis.

Validation (point de contrôle C). Deux exécutions successives du client :

```
1re execution : etat avant : inconnu
        etat apres (relu) : en observation
        observations : [Fievre 39.5]
2e execution :  etat avant : en observation
        observations : [Fievre 39.5, Fievre 39.5]
```

`Proxy.isProxyClass` renvoie `true` pour le dossier. La seconde exécution voit la modification de la première : le dossier est bien partagé et conservé côté serveur.

6. Étape A4 — Le cabinet : un point d'entrée unique

Implémentation. `ICabinet extends Remote` expose `getPatients()` et `rechercherPatient(String nom)`. `CabinetImpl` conserve la collection de patients ; `ajouterPatient` est une méthode **locale**, absente du contrat distant, que seul le lanceur utilise. Le serveur ne publie plus que `cabinet` dans le registre : on n'a plus besoin de connaître le nom de chaque patient à l'avance.

Décision — que retourne `getPatients()` ? Des **stubs**, pas des copies. C'est la conséquence directe de A1 : `AnimalImpl` est exporté, donc lors de la sérialisation de la liste, chaque élément exporté est remplacé par son stub. La liste elle-même est copiée, mais elle contient des moyens de joindre les animaux.

Décision — recherche infructueuse. `rechercherPatient` retourne `null` si aucun patient ne porte ce nom. Ce comportement est documenté dans le contrat et le client doit tester ce cas (affichage « introuvable »). Une exception métier aurait été possible ; `null` est plus simple pour une absence qui n'est pas une erreur.

Validation (point de contrôle D).

```
Liaisons dans le registre : [cabinet]
Nombre de patients : 2
 - Moon (chien) proxy ? true
 - Felix (chat) proxy ? true
Recherche 'Moon' : Moon, maitre Alex
Recherche 'Inconnu' : introuvable
```

7. Étape A5 — Créer un patient depuis le client

Problème. Un `new AnimalImpl(...)` exécuté dans la JVM du client ne créerait pas d'objet « chez le serveur » ; d'ailleurs le client ne connaît pas `AnimalImpl` (projet `server`).

Décision — quels paramètres traversent le réseau et sous quelle forme ? Le client n'envoie que des **données** ; l'animal est instancié par le serveur. Opération ajoutée à `ICabinet` :

```
IAnimal enregistrerPatient(String nom, String maitre, String race, Espece espece)
```

Élément	Type	Passage
nom, maître, race	`String`	par valeur
espèce	`Espece` (Serializable)	par valeur (copie)
résultat	`IAnimal`	stub : l'animal reste sur le serveur

L'espèce est transmise par valeur, en cohérence avec A2.

Validation (point de contrôle E) par interaction distante, non par trace serveur. Après l'appel, le nombre de patients renvoyé par le serveur passe de 2 à 3, et `rechercherPatient("Rex")` retrouve le patient. Une seconde exécution du client voit toujours 3 patients : l'état est créé et conservé côté serveur.

```
Patients avant ajout : 2
Patient cree : Rex (proxy ? true)
Patients apres ajout : 3
Recherche 'Rex' : Rex, maitre Paul
```

8. Étape A6 — Alertes : le serveur rappelle ses clients

Principe. C'est le patron Observer réalisé à travers le réseau : « client » et « serveur » sont des rôles. Le client héberge à son tour un objet distant que le serveur rappelle (callback).

Implémentation.

- `IObservateurCabinet extends Remote` (dans `common`) : `notifier(int seuil, int nombrePatients, boolean aLaHausse)`.
- `ObservateurAlertes` (dans `client`) l'implémente et est **exporté** par le client.
- `ICabinet` expose `sabonner` et `seDesabonner` : le serveur reçoit le **stub** de l'observateur et le range dans sa liste.
- `CabinetImpl` compare le nombre de patients avant et après chaque ajout ou retrait ; un seuil (100, 500, 1000) est franchi s'il se trouve entre les deux valeurs. Cela couvre la hausse comme la baisse.
- Une opération `retirerPatient` a été ajoutée au contrat pour pouvoir observer le franchissement à la baisse (le sujet ne la demande pas explicitement).

Décisions de robustesse (question du sujet : que se passe-t-il si un client abonné est fermé ?).

- Chaque notification est protégée par son propre `try/catch` : une `RemoteException` ne bloque pas les autres abonnés, et l'observateur injoignable est **retiré de la liste**.
- La notification travaille sur une **copie** de la liste : un abonnement pendant une notification ne la corrompt pas.
- La notification est faite **hors du verrou** de la liste des patients, pour qu'un client lent ne bloque pas les autres opérations.

Validation (point de contrôle F). Deux clients abonnés, un troisième (`ClientRemplisseur`) amenant le cabinet à 100 patients puis le ramenant à 99. Chaque client abonné affiche :

```
[ALERTE] seuil 100 franchi a la hausse : 100 patients
[ALERTE] seuil 100 franchi a la baisse : 99 patients
```

Après fermeture brutale (`Ctrl+C`) du premier client, le second continue de recevoir les alertes et le serveur affiche `[cabinet] observateur injoignable, retire : ConnectException`.

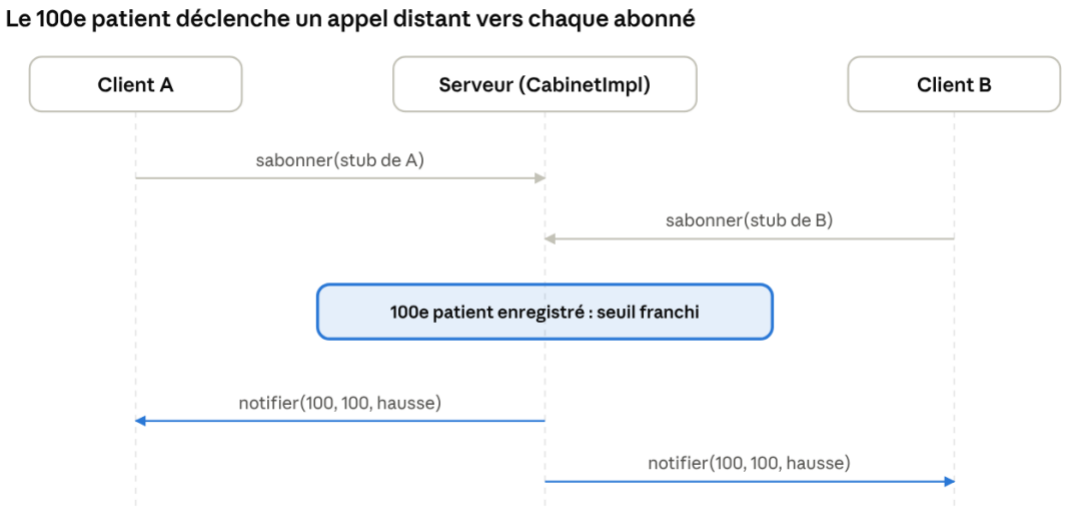


Schéma 3. Le serveur conserve le stub de chaque observateur ; au franchissement d'un seuil, chaque notification est un appel distant vers le client, qui peut échouer.

9. Étape A7 — Ce qui traverse vraiment le réseau

Expérience. Dans le projet `client` uniquement, une sous-classe `Chien` de `Espece` est définie (constructeur `super("Chien", 13)`), puis une instance est envoyée à `enregistrerPatient`.

1. Observation de l'échec. La cause profonde est lue en suivant la chaîne d'exceptions :

```
ServerException
 -> UnmarshalException : error unmarshalling arguments
  -> ClassNotFoundException : vete.client.Chien
    (no security manager: RMI class loader disabled)
```

2. Explication. Un flux sérialisé contient l'**état** de l'objet (les valeurs de ses champs) et le **nom** de sa classe. Il ne contient **pas** le bytecode de la classe. Pour reconstruire l'objet, la JVM du serveur doit charger elle-même `vete.client.Chien` depuis son propre classpath ; or cette classe n'existe que dans le projet `client`. Le serveur ne la trouve pas et lève `ClassNotFoundException`, que RMI remonte au client sous la forme `UnmarshalException` puis `ServerException`. Le chargement dynamique de classes de RMI est désactivé ici (ni `SecurityManager`, ni codebase), donc rien ne peut compenser.

3. Correction la plus simple. Déplacer `Chien` dans `common`, connu des deux côtés (le package devient `vete.common`). Après recompilation des trois projets et redémarrage du serveur, le patient est ajouté normalement.

Lien avec le reste du TP. C'est la même règle qu'en A2 : toute classe qui traverse le réseau par valeur doit être dans le classpath des deux extrémités ; c'est le rôle du projet `common`.

10. Étape A8 — Une application utilisable : la CLI

Organisation en trois responsabilités.

- `Main` (point d'entrée) lit les arguments (`hôte`, `port`, par défaut `localhost` et 1099), construit la logique client et lance l'interface.
- `ClientCabinet` (logique client) regroupe les opérations de l'application : lister, rechercher, enregistrer, consulter et modifier un dossier, s'abonner. Il est le seul à parler aux interfaces distantes. Il renvoie des résultats simples (des `String`, des `List<String>`), pas des stubs.
- `ConsoleUI` (interface console) affiche le menu, lit et valide les saisies avec `BufferedReader`. **Elle n'importe rien de `java.rmi`** : c'est vérifiable par `grep`.

Saisies invalides. L'interface refuse une option inconnue, un champ vide ou une espérance de vie non numérique, affiche un message et redemande, sans arrêter le programme. Une fin de flux (`Ctrl+D`) est traitée comme une demande de sortie.

Pannes distantes. `ClientCabinet` attrape les `RemoteException` et les convertit en `ErreurCabinet`, une exception métier portant un message lisible (par exemple `Serveur injoignable. Verifiez qu'il est demarre.`). L'interface n'a plus qu'à afficher ce message et le menu reste utilisable : le client ne s'arrête pas.

Alertes et arrêt propre. Les alertes arrivent sur un fil du runtime RMI. La logique client les transmet à l'interface par une fonction d'affichage injectée (`Consumer<String>`), sans que l'interface connaisse RMI ; elles sont préfixées `[ALERTE]` pour se distinguer du menu. À la sortie, `fermer()` retire l'observateur du runtime RMI (`UnicastRemoteObject.unexportObject(obs, true)`) : sans cela, l'objet exporté garde la JVM en vie et le programme ne se terminerait pas.

Validation (point de contrôle H). Avec deux clients CLI simultanés : un patient enregistré par l'un est trouvé par l'autre ; une modification du dossier de suivi est visible depuis l'autre client ; les deux clients abonnés reçoivent l'alerte ; les saisies invalides sont refusées ; serveur arrêté, le client affiche « Serveur injoignable » ; la sortie par `0` rend la main au terminal.

11. Synthèse

Ce qui fonctionne. Les huit points de contrôle (0, A à H) sont franchis avec des sorties observées. L'application tourne en processus séparés (un serveur, au moins deux clients CLI) ; le client ne dépend que de `common` ; il n'y a pas de `module-info.java`.

Ce qui ne fonctionne pas, ou pas encore. Les données sont en mémoire : redémarrer le serveur remet l'état initial (deux patients). Un client qui a perdu son serveur doit être relancé pour retrouver un stub valide.

Difficultés rencontrées et résolution.

Difficulté	Cause	Solution
Le client voit un ancien comportement	Serveur démarré avant la modification du code	Redémarrer le serveur après chaque changement, dans cet ordre : serveur puis clients
`ClassNotFoundException: vete.server.Server`	Mauvaise arborescence (répertoire `src` imbriqué,	Aligner paquets et dossiers (`vete/common`, etc.) et recompiler avec `-d`

Difficulté	Cause	Solution
	paquet qui ne correspond pas au dossier)	
Échec en A7	Classe `Chien` connue du seul client	La déplacer dans `common` (voir A7)
Un programme de test ne compilait plus	Le contrat `ICabinet` avait changé en A6	Supprimer le client de test devenu inutile (`ClientAbonne`), remplacé par la CLI

Le débogage d'une application distribuée a confirmé le conseil du sujet : ne passer à l'étape suivante qu'après avoir franchi le point de contrôle de la précédente.

12. Regard critique

- **Persistance.** Remplacer les collections en mémoire par une base de données ; un cabinet réel ne peut pas perdre ses dossiers à chaque redémarrage.
- **Sécurité.** Aucune authentification ni chiffrement : n'importe quel client joignable peut lire et modifier les dossiers. La sérialisation Java d'objets venant du réseau est aussi un risque connu (filtrage de classes à prévoir).
- **Passage à l'échelle.** `getPatients()` renvoie un stub par patient : avec des milliers de patients, mieux vaudrait une pagination ou une recherche côté serveur.
- **Callbacks et réseau.** Le serveur doit pouvoir joindre le client : c'est fragile derrière un pare-feu ou un NAT, et la détection d'un client mort n'a lieu qu'à la prochaine notification (pas de bail ni de ping).
- **Seuils.** Les seuils 100, 500 et 1000 sont figés dans le code ; ils pourraient être paramétrables par abonné.
- **Tests.** Les validations sont manuelles (programmes de test et sorties observées) ; des tests automatisés (JUnit avec un registre sur un port libre) éviteraient les régressions.
- **Couplage à Java.** RMI suppose Java des deux côtés et des classes partagées. C'est la limite qui motive les technologies suivantes du cours (SOAP, REST, gRPC), indépendantes du langage.

13. Usage des outils d'intelligence artificielle générative

Outil utilisé. J'ai utilisé l'assistant Claude (Anthropic) tout au long du TP, de A0 à A8 et pour la rédaction du README et de ce compte-rendu.

Pour quelles tâches et avec quel type de sollicitations.

- **Comprendre le cours et le sujet** : expliquer les notions (stub, objet distant ou sérialisable, registre, Observer, sérialisation), reformuler les étapes du sujet en étapes plus petites, expliquer pourquoi la structure `common` / `server` / `client` est utilisée. Étudiant salarié, je suis le cours à distance : cet appui m'a été particulièrement utile.

- **Guidage pas à pas** : j'ai demandé qu'on m'explique d'abord l'architecture de chaque étape, puis qu'on avance ensemble ; l'assistant m'a indiqué quels fichiers créer et quelles commandes `javac` / `java` lancer. Une grande partie de ces indications figurait déjà dans les supports de cours : l'assistant m'a surtout orienté vers les bons documents, et la démarche suivie reste celle du cours.
- **Compréhension des erreurs** : j'ai fourni les messages de compilation et d'exécution (paquets mal placés, imports, `ClassNotFoundException`, serveur non redémarré) et l'assistant en a expliqué les causes.
- **Alternatives de conception** : discussion sur l'objet distant ou sérialisable pour le dossier de suivi (A3), sur `null` ou exception pour une recherche infructueuse (A4), sur la gestion des observateurs injoignables (A6).
- **Rédaction** : rédaction du README et du brouillon de ce compte-rendu avec l'assistant ; je les ai lus, corrigés et complétés.

Ce qui a réellement influencé le rendu.

- J'ai écrit et exécuté le code moi-même en suivant la structure expliquée par l'assistant ; pour les parties répétitives ou détaillées, le code vient de l'assistant, notamment les ajouts de l'étape A6 (`retirerPatient`, retrait des observateurs injoignables, notification sur une copie de la liste et hors verrou).
- Pour l'étape A8, la première version des classes `ConsoleUI` et `Main` a été rédigée par l'assistant ; l'organisation en trois couches suit la consigne du sujet. J'ai volontairement confié à l'assistant l'essentiel de la charge de code de cette étape, que j'ai ensuite lue et testée.
- L'explication de l'expérience A7 et les brouillons du README et de ce rapport ont été rédigés avec l'assistant.

Comment je l'ai vérifié, corrigé ou adapté.

- J'ai compilé et exécuté chaque étape moi-même en processus séparés et franchi les points de contrôle avec des sorties réelles, reproduites dans ce rapport.
- J'ai corrigé moi-même les erreurs d'organisation (arborescence, paquets, imports, classe `Chien` déplacée).
- J'ai testé la CLI avec deux clients simultanés, des saisies invalides, un serveur arrêté et la sortie propre.
- Les contrôles de fichiers et de dépendances (`grep`, compilation) ont été lancés en partie par moi, en partie par l'assistant à ma demande ; j'ai vérifié leurs sorties.

Ce qui est entièrement de moi : la mise en place de l'environnement de travail, l'exécution de chaque étape et de tous les tests, la correction des erreurs, la vidéo de démonstration, ainsi que la relecture et la correction finales du README et du compte-rendu.

Je peux expliquer chaque élément significatif de ce travail.

14. Références

1. Bachar Rima, *HAI704I — CM01 : Introduction*, Université de Montpellier, 2026–2027.
2. Bachar Rima, *HAI704I — CM02 : RMI*, Université de Montpellier, 2026–2027.
3. *HAI704I — TD1 : RMI* et sa correction, 2026–2027.
4. *HAI704I — TP1 : RMI* et annexes (Workflow Hints, Patrons utiles, Spring Boot), 2026–2027.
5. Oracle, *Java SE Documentation — package* `java.rmi` et *Java Object Serialization Specification*.
6. Git : https://github.com/serdarvarl/HAI704I-TP1-RMI-cabinet-veterinaire.git
7. Vidéo de démonstration : demonstration_Varol_Serdar.mp4